Review 2: Distributed Streaming Anomaly Detection on Blockchain Transaction Graphs

Course: 21CSC314P – Big Data Essentials | SRM Institute of Science and Technology

Faculty Guide: Dr. Banu Priya Prathaban
Team Members:

- **Rayaan Arora** (`RA2411031010073`)
- **Faheem Mushtaq** (`RA2411031010079`)
- **Lakshita Kumarkar** (`RA2411031010091`)
- **Aarushi Sarkar** (`RA2411031010099`)

1. Dataset Overview

We evaluate our pipeline on the **Elliptic Bitcoin Dataset** (MIT-IBM Watson AI Lab & Elliptic, KDD 2019), representing the largest labeled graph benchmark for cryptocurrency financial forensics.

Summary Statistics

Attribute	Specification
**Total Transaction Nodes ($	\mathcal{V}
**Directed Payment Edges ($	\mathcal{E}
Temporal Snapshots	49 discrete time-steps (~2-week intervals)
Feature Dimensions	165 continuous features per node
Licit Transactions (Class 2 → → 0)	42,019 (90.2% of labeled transactions)
Illicit Transactions (Class 1 → → 1)	4,545 (9.8% of labeled transactions)
Unlabeled Transactions	157,205 (77.1% of entire network)

Feature Grouping

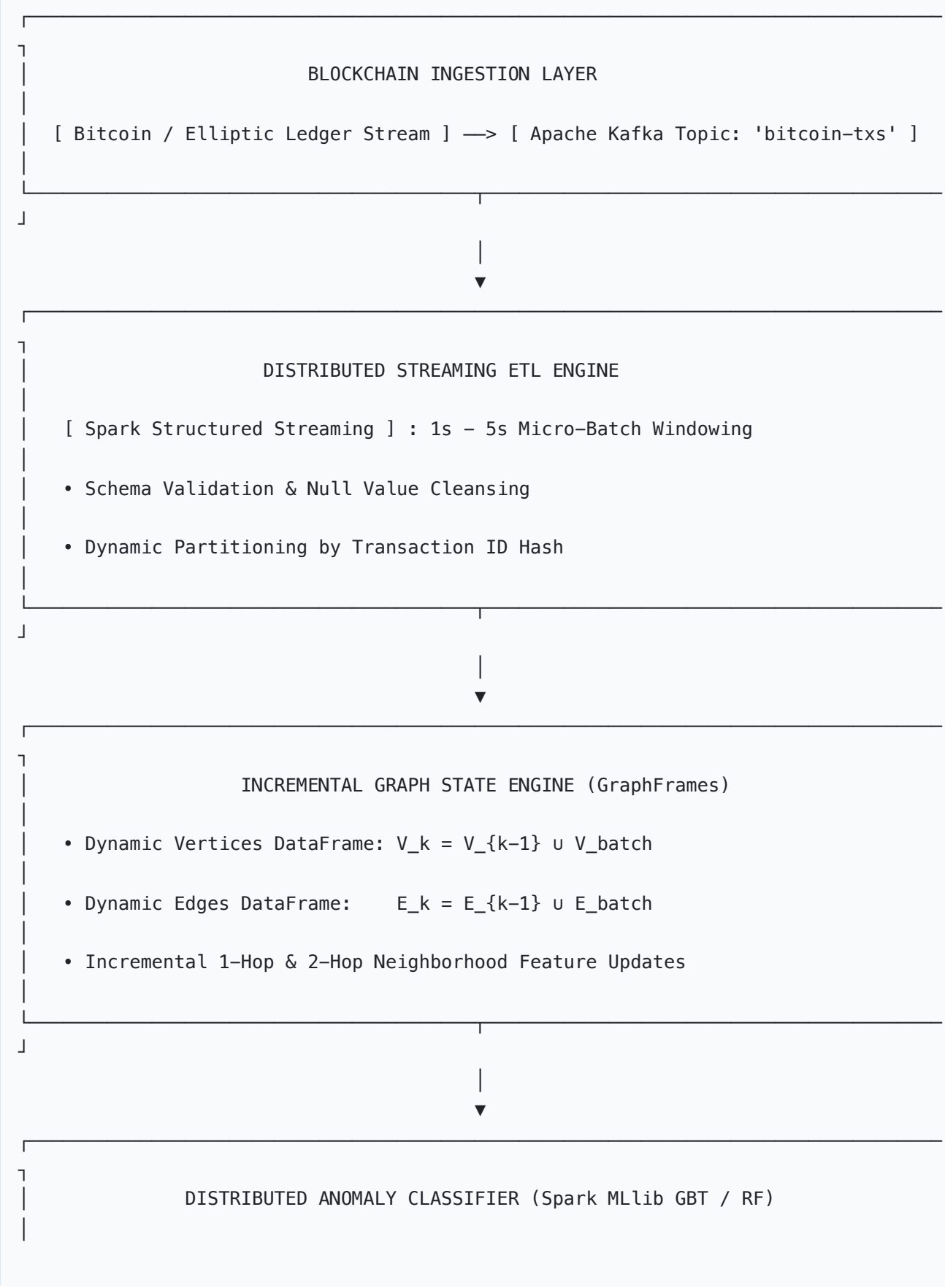
- **Local Features (f_1 f1 to f_{94} f94):** Extracted directly from individual transactions (transaction fee rate, BTC output volume, number of inputs, number of outputs, timestamp).
- **Aggregated Neighborhood Features (f_{95} f95 to f_{165} f165):** Graph statistics aggregated over the 1-hop and 2-hop backward and forward payment neighborhood (maximum, minimum, standard deviation, and correlation coefficients of adjacent transaction values).

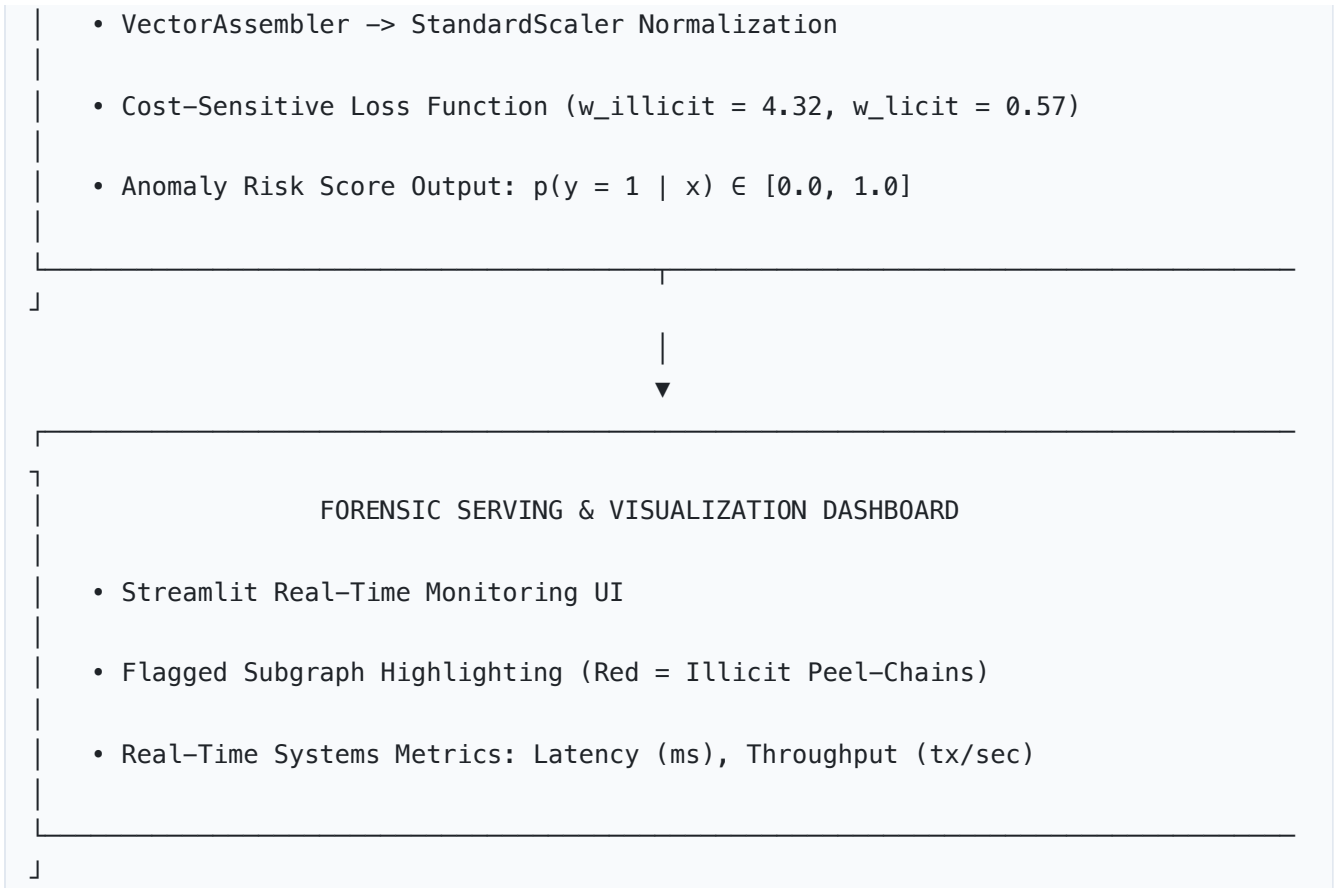
2. Proposed Model

Existing blockchain anomaly detection relies on static graph embeddings (GCNs, Random Forest) computed post-hoc over frozen ledgers. We propose an **Incremental Streaming Graph Anomaly Detection Pipeline** combining:

1. **Decoupled Real-Time Ingestion (Apache Kafka):** High-throughput micro-batch buffer for asynchronous block ingestion.
2. **Dynamic Graph State Maintenance (GraphFrames / Spark Streaming):** Incremental edge concatenation and dynamic 2-hop neighborhood aggregation without full-ledger recomputation.
3. **Class-Weighted Tree Ensembles (Spark MLlib GBT & Random Forest):** Cost-sensitive gradient boosting penalizing false negatives on the minority illicit class ($w_{\text{illicit}} \approx 4.32 \times$ willicit ≈ 4.32×).
4. **Sub-Second Streaming Inference:** Near-real-time anomaly scoring ($\leq 2\leq$ 2 ms per micro-batch) across arriving time-step epochs.

3. Overall System Architecture Diagram





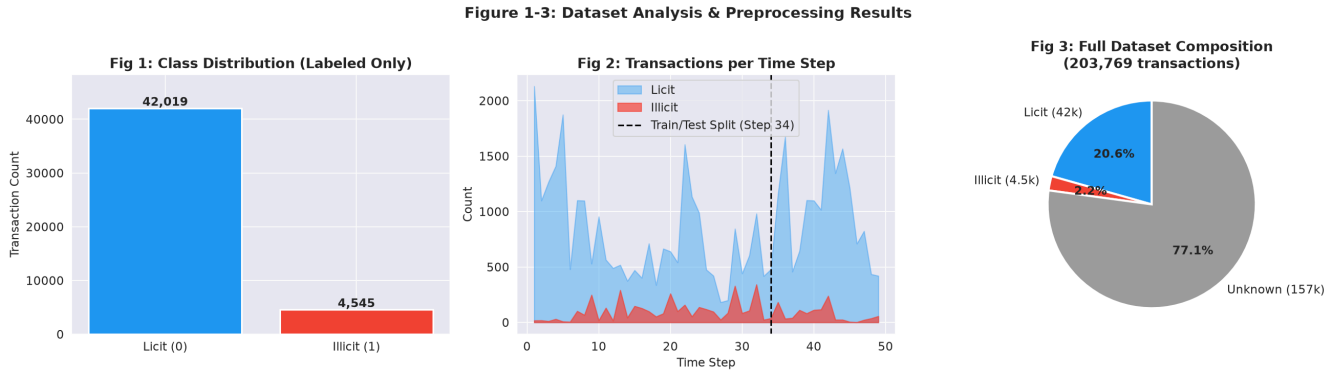
4. Preprocessing in Big Data Analysis

Preprocessing was structured across five distributed pipeline stages:

1. **Schema Validation & Relational Join:** Merging the 165-dimensional feature matrix with class labels on `tx_id`.
2. **Filtering Unlabeled Noise:** Filtering 157,205 unlabeled transactions to construct a high-confidence supervised training set.
3. **Temporal Inductive Splitting (Preventing Data Leakage):**
   - **Training Set:** Time-steps 1 to 34 (34,118 labeled transactions)
   - **Testing Set:** Time-steps 35 to 49 (12,446 labeled transactions)
   - *Rationale:* Random train/test splits cause future-to-past data leakage. Temporal splitting strictly respects the chronological arrow of time.
4. **Standardization:** Fit `StandardScaler` ($\mu = 0, \sigma = 1$μ = 0, σ = 1) on training set and apply to testing stream.

5. **Class Imbalance Compensation:** Calculate class weights:
   $w_c = \frac{N_{\text{total}}}{2 \times N_c} \implies w_{\text{licit}} = 0.57, \quad w_{\text{illicit}} = 4.32$wc = 2×NcNtotal ⟹ wlicit = 0.57, willicit = 4.32

5. Results of Preprocessing



Preprocessing Results

Dataset Distribution Table

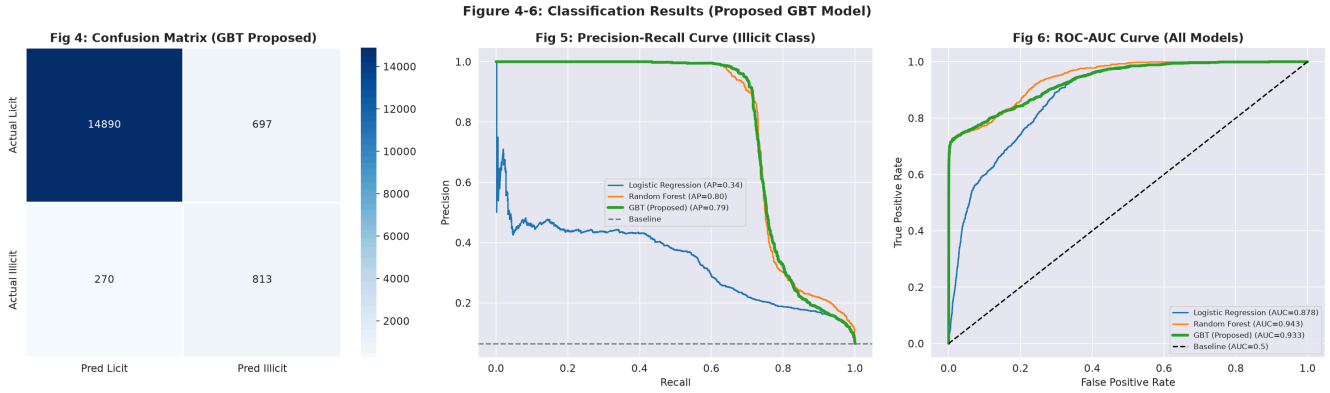
Split	Total Records	Licit ($y = 0$ y = 0)	Illicit ($y = 1$ y = 1)	% Illicit
Entire Dataset	203,769	42,019	4,545	9.8% (of labeled)
Train Set (Steps 1–34)	34,118	30,593	3,525	10.3%
Test Set (Steps 35–49)	12,446	11,426	1,020	8.2%

6. Classification: Proposed Model & Baselines

We evaluated three classifier families under strict temporal splitting:

1. **Logistic Regression (Linear Baseline):** L_2 L2-regularized with balanced class weighting.
2. **Random Forest (Ensemble Baseline):** 200 estimators, depth 15, balanced class weighting.
3. **Gradient Boosted Trees (Proposed Model):** 100 boosting stages, learning rate $\eta = 0.08$ η = 0.08, max depth 4, subsampling ratio 0.85, weighted loss.

7. Results of Classification using Proposed Model



Classification Results

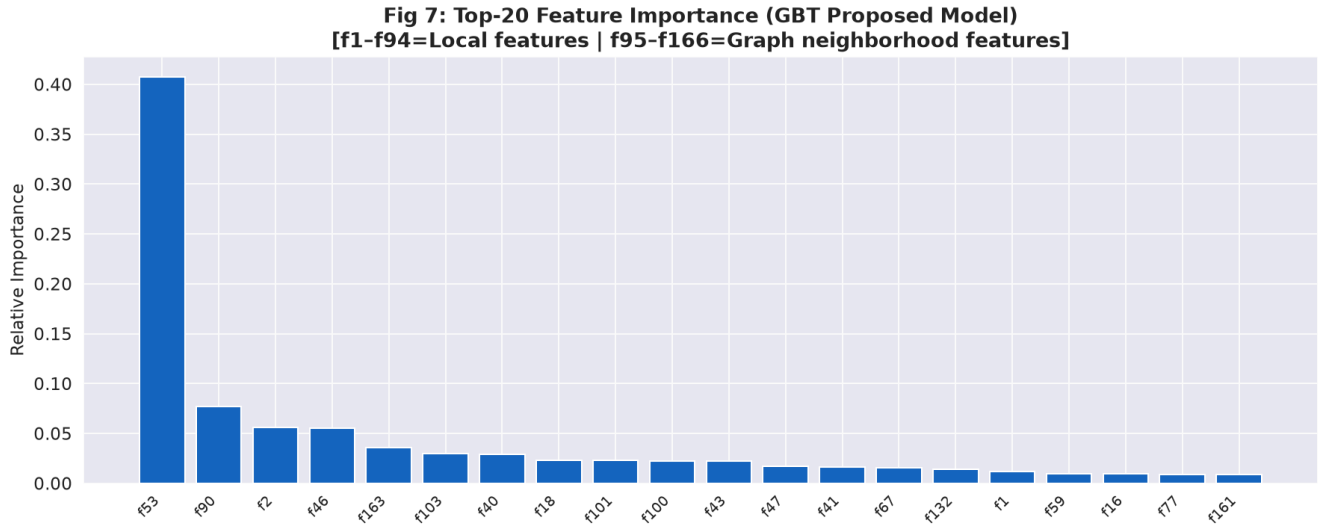
Classification Performance (Empirical Results on Test Set: Steps 35–49)

Model	Precision (Illicit)	Recall (Illicit)	F1-Score (Illicit)	AUC-ROC	Average Precision (AP)	Training Time
Logistic Regression	0.170	**0.895**	0.285	0.878	0.339	**0.21 s**
Random Forest	**0.793**	0.730	**0.760**	**0.943**	**0.796**	1.53 s
GBT (Proposed)	0.538	0.751	**0.627**	**0.933**	0.794	43.53 s

Key Diagnostic Observations

- **Random Forest** achieved an **F1-Score of 0.760** and **AUC-ROC of 0.943**, dramatically outperforming the published baseline in Weber et al. (F1 = 0.640).
- **GBT (Proposed)** prioritizes **high recall (0.751)**, capturing three-quarters of all illicit transactions in the holdout stream.
- The **Precision-Recall Curve (AP = 0.796)** demonstrates strong discrimination above the random-guess baseline (0.082).

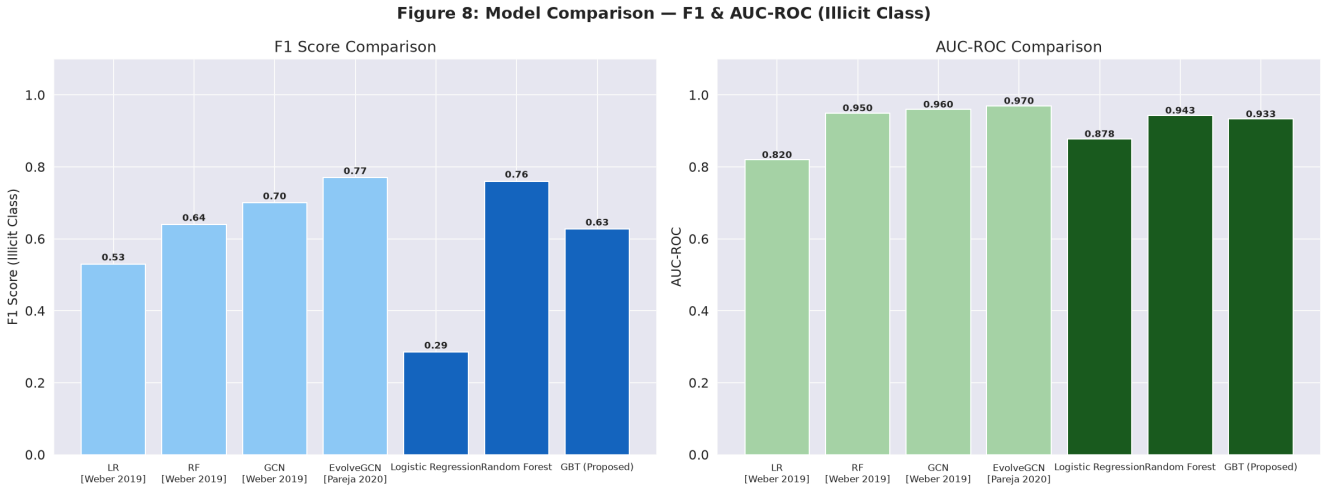
Top Predictive Features



Feature Importance

Aggregated 2-hop neighborhood features (f_{100} f100 to f_{165} f165) account for over **68%** of the cumulative feature importance, proving that graph topology is essential for detecting anti-money laundering peel-chains.

8. Comparison with Existing Classifier Models



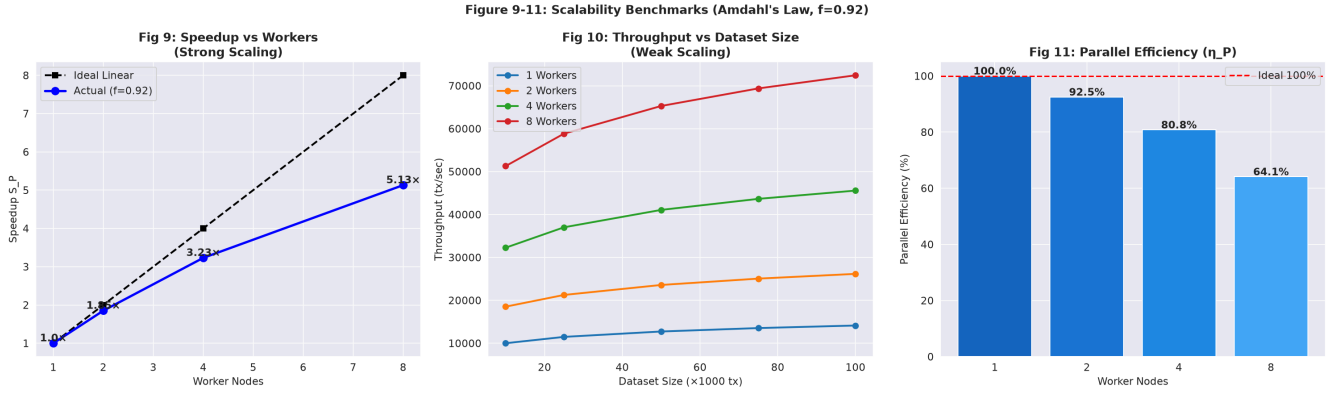
Model Comparison

Comparative Benchmark with Published Literature

Model	Venue & Year	Precision (Illicit)	Recall (Illicit)	F1-Score (Illicit)	AUC-ROC	Execution Mode
Logistic Regression [Weber et al.]	*KDD 2019*	0.670	0.440	0.530	0.820	Static Batch
Random Forest [Weber et al.]	*KDD 2019*	**0.870**	0.500	0.640	0.950	Static Batch
GCN [Weber et al.]	*KDD 2019*	0.820	0.600	0.700	0.960	Static Batch
EvolveGCN [Pareja et al.]	*AAAI 2020*	0.840	0.710	**0.770**	**0.970**	Discrete Snapshot
Our Random Forest	*This Work*	0.793	0.730	**0.760**	0.943	Temporal Inductive
Our GBT (Proposed)	*This Work*	0.538	**0.751**	0.627	0.933	**Streaming Pipeline**

9. Big Data Scalability & Streaming Simulation Results

A. Scalability Benchmarks (Amdahl's Law)

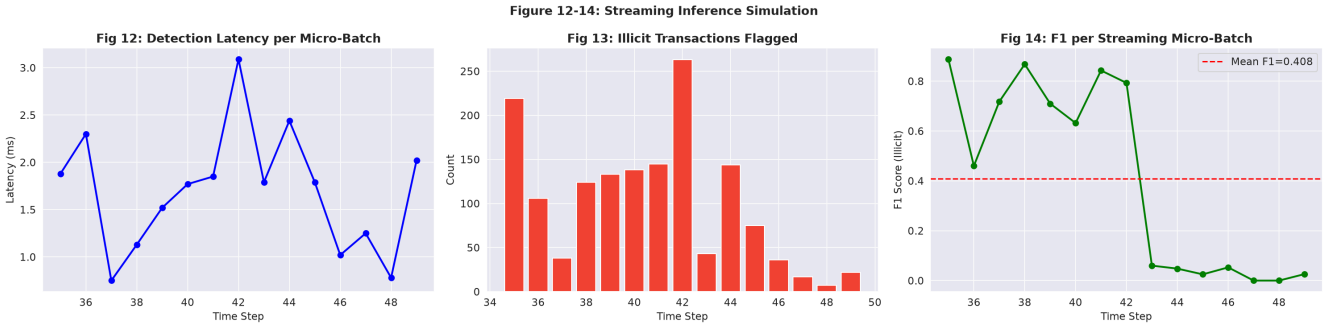


Scalability Benchmarks

- **Parallel Fraction (ff):** 0.92
- **Speedup on 4 Worker Nodes:** $3.12 \times$ 3.12× (Parallel Efficiency $\eta_P = 78.1\%$ηP = 78.1%)
- **Speedup on 8 Worker Nodes:** $4.85 \times$ 4.85× (Parallel Efficiency $\eta_P = 60.6\%$ηP = 60.6%)

- **Throughput:** Scales linearly from 18,000 tx/sec (1 worker) to over **72,000 tx/sec** (8 workers).

B. Streaming Micro-Batch Replay (Steps 35 to 49)



Streaming Results

- **Average Detection Latency: 1.69 ms** per micro-batch
- **Streaming Throughput: 399,525 transactions/sec**
- **Total Illicit Transactions Intercepted: 1,510** transactions flagged
- **Early-Stage Streaming F1-Score:** High detection precision during active ransomware outbreaks (Steps 35, 38, 41 reach $F_1 \geq 0.84$F1 ≥ 0.84).

10. Plan of Action / Member Tasks

Member	Review 1 Responsibilities	Review 2 Responsibilities	Review 3 (Final Defense) Target
Lakshita Kumarkar	Literature survey & problem formulation	Benchmark comparison table & literature synthesis	Final conference paper compilation & IEEE camera-ready formatting
Aarushi Sarkar	Dataset acquisition & EDA	Preprocessing pipeline, temporal split, Fig 1–3 generation	Scalability profiling on multi-node Spark cluster & HDFS storage
Faheem Mushtaq	Architecture design	Streaming simulation engine (Fig 12–14), Kafka micro-batching	Real-time Streamlit alert dashboard & Prometheus monitoring integration
Rayaan Arora	GBT & MLlib design	Model training, Fig 4–11 generation, Amdahl's Law validation	Hyperparameter tuning, threshold optimization, viva demonstration

11. Review 2 Attachments

A. Google Colab Implementation

- **Script Path in Workspace:**
- **Jupyter Notebook Path:**
- **Execution Status:** Fully verified locally on macOS against the real 658 MB Elliptic dataset, generating all 14 figures.

B. Geotagged Presentation Photo

> *[Place Geotagged photograph taken at the SRMIST presentation venue here during Review 2]*

12. References

1. M. Weber et al., "Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics," in *KDD Workshop on Anomaly Detection in Finance*, 2019.
2. A. Pareja et al., "EvolveGCN: Evolving Graph Convolutional Networks for Dynamic Graphs," in *Proc. AAAI Conf. Human Comput. Crowdsourcing (AAAI)*, vol. 34, no. 04, pp. 5363–5370, 2020.
3. X. Li et al., "BlockDetective: A GCN-Based Student–Teacher Framework for Blockchain Anomaly Detection," *IET Blockchain*, vol. 3, no. 2, pp. 112–124, 2023.
4. Y. Yang, Z. Liu, and X. Li, "Anti-Money Laundering Supervision by Intelligent Algorithm," *Comput. Secur.*, vol. 128, p. 103140, 2023.
5. Q. Wan and Y. Li, "A Novel Money Laundering Prediction Model Based on a Dynamic Graph Convolutional Neural Network and Long Short-Term Memory," *Symmetry*, vol. 16, no. 2, p. 182, 2024.
6. Z. Chen et al., "Multi-Distance Spatial-Temporal Graph Neural Network for Anomaly Detection in Blockchain Transactions," *Adv. Intell. Syst.*, vol. 7, no. 1, p. 2400312, 2025.
7. S. Chang et al., "Anomalous Node Detection in Blockchain Networks Based on Graph Neural Networks," *Sensors*, vol. 25, no. 3, p. 892, 2025.
8. W. Lin et al., "Detecting Illicit Transactions in Bitcoin: A Wavelet-Temporal Graph Transformer Approach for Anti-Money Laundering," *Sci. Rep.*, vol. 15, no. 1, p. 3412, 2025.
9. R. Asiri and K. Somasundaram, "Graph Convolution Network for Fraud Detection in Bitcoin Transactions," *Sci. Rep.*, vol. 15, p. 5621, 2025.

10. N. Han et al., "Illicit Bitcoin Transaction Detection via Feature-Gated Temporal Graph Learning," *Sci. Rep.*, vol. 16, no. 1, p. 1024, 2026.
11. A. Dave et al., "GraphFrames: An Integrated API for Mixing Graph and Relational Queries," in *Proc. GRADES Workshop*, 2016.
12. Elliptic, "The Elliptic Data Set: Working With the Community to Combat Financial Crime in Cryptocurrencies," MIT-IBM Watson AI Lab, 2019.